

TOPIC 08.0: QUEUE FOUNDATION & FIFO PRINCIPLE

08.0 QUEUE FOUNDATION & FIFO PRINCIPLE →

TOPIC 08.0: QUEUE FOUNDATION & FIFO PRINCIPLE

PAGE 1 OF 10

See Topic 07 → Stack Masterclass

GOAL: Master FIFO (First In, First Out) thinking, understand Double-Ended Queues (Deque), and master Monotonic Deque for Sliding Window Maximum in O(N) time!

REUSABLE QUEUE & DEQUE UTILITIES

Queue & Deque Operation Reference:

```
// Queue (FIFO): offer(), poll(), peek()
Queue<Integer> q = new ArrayDeque<>();
q.offer(val); int front = q.poll();

// Deque (Double Ended): offerFirst(), offerLast(), pollFirst(), pollLast()
Deque<Integer> deque = new ArrayDeque<>();
```

Circular Queue Modulo Math:

```
// Modulo Arithmetic for Circular Indexing
int nextIndex = (currIndex + 1) % capacity;
int prevIndex = (currIndex - 1 + capacity) % capacity;
```

AHA MOMENT #1: FIRST IN, FIRST OUT (FIFO) ORDER PRESERVATION
A Queue is like a line at a supermarket checkout: elements enter at the rear (`enqueue / offer`) and leave from the front (`dequeue / poll`). Unlike Stack which reverses arrival order, Queue **preserves the exact arrival order**, making it the foundational engine for BFS graph traversals and streaming buffers!

CORE QUEUE OPERATIONS (O(1) TIME)

- `offer(x)`: Adds element 'x' to rear of queue (O(1)).
- `poll()`: Removes and returns front element (O(1)).
- `peek()`: Views front element without removing (O(1)).
- `isEmpty()`: Returns true if queue size is 0 (O(1)).

2 QUEUE VS STACK PARADIGM COMPARISON

Feature	Queue (FIFO)	Stack (LIFO)
Processing Order	First In, First Out	Last In, First Out
Primary Traversal	BFS (Level Order)	DFS (Depth First Search)
Entry/Exit Point	Enter Rear, Exit Front	Enter Top, Exit Top

VISUALIZING QUEUE ENQUEUE / DEQUEUE

1. OFFER(10), OFFER(20), OFFER(30): FRONT [10 → 20 → 30] REAR 2. POLL() → Removes 10 (First Element): FRONT [20 → 30] REAR

KEY TAKEAWAY: Use Queue whenever you must process requests in order of arrival or explore graph nodes layer-by-layer!

CIRCULAR QUEUE & JAVA QUEUE IMPLEMENTATIONS →

TOPIC 08.1: CIRCULAR QUEUE & JAVA IMPLEMENTATIONS

PAGE 2 OF 10

AHA MOMENT #2: CIRCULAR ARRAY MODULO ARITHMETIC
In a standard fixed array, dequeuing elements leaves empty spaces at index 0. Shifting remaining items costs O(N)! A **Circular Queue** re-uses empty spots using wrap-around modulo arithmetic:
`head = (head + 1) % capacity` and `tail = (tail + 1) % capacity` in O(1) time!

CIRCULAR ARRAY QUEUE MECHANICS

```
class CircularQueue {
    private int[] data;
    private int head = 0, tail = 0, size = 0, capacity;
    public CircularQueue(int k) {
        capacity = k; data = new int[k];
    }
    public boolean enqueue(int value) {
        if (isFull()) return false;
        data[tail] = value;
        tail = (tail + 1) % capacity; // Modulo wrap
        size++; return true;
    }
    public boolean dequeue() {
        if (isEmpty()) return false;
        head = (head + 1) % capacity; // Modulo wrap
        size--; return true;
    }
    public boolean isEmpty() { return size == 0; }
    public boolean isFull() { return size == capacity; }
}
```

2 ARRAYDEQUE VS LINKEDLIST QUEUE

Feature	ArrayDeque	LinkedList
Memory Overhead	Contiguous Array (Low)	Heap Node Objects (High)
Cache Locality	High (CPU Cache friendly)	Low (Pointer chasing)
Null Values	Disallows <code>null</code>	Allows <code>null</code>
FAANG Verdict	✅ PREFERRED CHOICE	⚠️ Avoid unless null needed

JAVA API TIP: Prefer `offer()`, `poll()`, `peek()` over `add()`, `remove()`, `element()` because offer/poll return `false / null` instead of throwing exceptions!

TOPIC 08.0: QUEUE FOUNDATION & FIFO PRINCIPLE

DEQUE FOUNDATION (DOUBLE-ENDED QUEUE) →

⬅️ **TOPIC 08.2: DEQUE FOUNDATION (DOUBLE-ENDED QUEUE)** PAGE 3 OF 10

💡 **AHA MOMENT #3: DUAL-ENDED POWER (FRONT AND BACK ACCESS)**
A **Deque** (Double-Ended Queue) allows inserting and deleting from BOTH the front AND the back in O(1) time! This versatility allows a Deque to act as a FIFO Queue, a LIFO Stack, or a **Monotonic Deque** for Sliding Window Maximum!

📋 **DEQUE METHOD MAPPING CHEAT SHEET**

Operation	Front Method	Back Method	Equivalent Stack/Queue Usage
Insert	addFirst(e) / offerFirst(e)	addLast(e) / offerLast(e)	Queue Enqueue: offerLast(), Stack Push: push() / addFirst()
Remove	removeFirst() / pollFirst()	removeLast() / pollLast()	Queue Dequeue: pollFirst(), Stack Pop: pop() / removeFirst()
Examine	getFirst() / peekFirst()	getLast() / peekLast()	Queue Front: peekFirst(), Stack Top: peek() / peekFirst()

🔗 **VISUALIZING DOUBLE-ENDED QUEUE (DEQUE)**

offerFirst(5) → | ← offerLast(20) | 5 → 10 → 15 → 20 | pollFirst() ← | → pollLast()

🔑 **KEY TAKEAWAY:** Always declare `Deque<Integer> deque = new ArrayDeque<>();` for maximum efficiency!

MONOTONIC DEQUE MASTERCLASS (SLIDING WINDOW MAX) →

📌 **TOPIC 08.3: MONOTONIC DEQUE MASTERCLASS (SLIDING WINDOW MAX)** PAGE 4 OF 10

💡 **AHA MOMENT #4: MONOTONIC DECREASING DEQUE (O(N) SLIDING WINDOW MAX)**
To find the Maximum in a Sliding Window of size K in O(N) total time, maintain a **Monotonic Decreasing Deque** of indices:
1. **Expire Old Elements**: If `deque.peekFirst() ≤ i - K`, pop from FRONT!
2. **Maintain Decreasing Order**: While `!deque.isEmpty() && arr[i] ≥ arr[deque.peekLast()]`, pop from BACK!
3. Maximum for current window is ALWAYS at `deque.peekFirst()`!

📋 **STEP-BY-STEP DEQUE STATE TRACE: WINDOW = [1, 3, -1, -3, 5], K = 3**

Read i	Value nums[i]	Monotonic Deque Action	Deque State (Indices / Values)	Window Max (peekFirst)
0	1	Push index 0 → Deque: [0]	[1]	N/A (i < K-1\$)
1	3	3 > 1 → Pop 0 from BACK. Push 1 → Deque: [1]	[3]	N/A (i < K-1\$)
2	-1	-1 < 3 → Push 2 → Deque: [1, 2]	[3, -1]	3 (at idx 1)
3	-3	-3 < -1 → Push 3 → Deque: [1, 2, 3]	[3, -1, -3]	3 (at idx 1)
4	5	Expire idx 1 (1 ≤ 4-3). 5 > -3, 5 > -1, 5 > 3 → Pop all from BACK! Push 4	[5]	5 (at idx 4)

🔑 **KEY TAKEAWAY:** Monotonic Deque achieves O(N) runtime because each element is pushed once and popped at most once!

TOPIC 08.0: QUEUE FOUNDATION & FIFO PRINCIPLE

BFS PREVIEW, DECISION TREE & PRINTABLE CHEAT SHEET →

 **TOPIC 08.4, 08.5 & 1-PAGE PRINTABLE CHEAT SHEET** PAGE 5 OF 10

 **QUEUE & DEQUE INTERVIEW DECISION TREE (MERMAID)**

 **PRINTABLE QUEUE & DEQUE CHEAT SHEET**

Pattern Type	Target Problem	Data Structure	Key Code Mechanism
Standard FIFO Queue	Recent Calls (LC 933)	<code>Queue<Integer> q = new ArrayDeque<>()</code>	<code>q.offer(val)</code> , <code>q.poll()</code>
Circular Queue	Design Circular Queue (LC 622)	Array + Modulo Pointers	<code>head = (head + 1) % K</code>
Sliding Window Max	Sliding Window Max (LC 239)	Monotonic Decreasing Deque	Pop smaller from back, pop expired from front
BFS Layer Processing	Level Order Traversal	Queue	<code>int size = q.size(); for(int i=0; i<size; i++)</code>
Two-Queue Voting	Dota2 Senate (LC 649)	2 Queues (Radiant, Dire)	Compare front indices, re-enqueue winner with <code>+N</code> offset

 **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!

EASY NEETCODE SUITE (LC 232, LC 933, MOVING AVERAGE) →

 **EASY NEETCODE: QUEUE USING STACKS, RECENT CALLS & MOVING AVG** PAGE 6 OF 10

1. LEETCODE 232 — IMPLEMENT QUEUE USING STACKS

 **Easy ★★★★★** **Two Stacks** **Amazon** **Microsoft**

Problem:
Implement FIFO queue using two stacks ('input' & 'output'). Amortized O(1) time!

```
class MyQueue {
    private Deque<Integer> in = new ArrayDeque<>();
    private Deque<Integer> out = new ArrayDeque<>();
    public void push(int x) { in.push(x); }
    public int pop() { peek(); return out.pop(); }
    public int peek() {
        if (out.isEmpty()) {
            while (!in.isEmpty()) out.push(in.pop()); // Reverse order
        }
        return out.peek();
    }
    public boolean empty() { return in.isEmpty() && out.isEmpty(); }
}
```

2. LEETCODE 933 — NUMBER OF RECENT CALLS

 **Easy ★★★★★** **Queue Range Expiry** **Google**

Problem:
Count number of recent requests within time range '[t - 3000, t]'.

```
class RecentCounter {
    private Queue<Integer> q = new ArrayDeque<>();
    public int ping(int t) {
        q.offer(t);
        while (!q.isEmpty() && q.peek() < t - 3000) {
            q.poll(); // Expire timestamps older than t - 3000
        }
        return q.size();
    }
}
```

 **AHA MOMENT (LC 232):** Two LIFO reversals cancel out to form a perfect FIFO Queue!

💡 **AHA MOMENT (LC 239):** Monotonic Deque cuts time complexity from $O(N \cdot K)$ brute-force to $O(N)$ optimal!

TOPIC 08.0: QUEUE FOUNDATION & FIFO PRINCIPLE

MEDIUM NEETCODE (DOTA2 SENATE & REVEAL CARDS) →

 MEDIUM NEETCODE: DOTA2 SENATE & REVEAL CARDS ORDER PAGE 8 OF 10

5. LEETCODE 649 — DOTA2 SENATE

Medium ★★★★★ Two Queues Simulation [Uber](#)

Problem:

Predict winning Senate party ('Radiant' vs 'Dire') in voting round simulation.

```
public String predictPartyVictory(String senate) {
    int n = senate.length();
    Queue<Integer> rad = new ArrayDeque<>(), dir = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        if (senate.charAt(i) == 'R') rad.offer(i);
        else dir.offer(i);
    }
    while (!rad.isEmpty() && !dir.isEmpty()) {
        int rIdx = rad.poll(), dIdx = dir.poll();
        if (rIdx < dIdx) rad.offer(rIdx + n); // Radiant bans Dire!
        else dir.offer(dIdx + n); // Dire bans Radiant!
    }
    return rad.isEmpty() ? "Dire" : "Radiant";
}
```

6. LEETCODE 950 — REVEAL CARDS IN INCREASING ORDER

Medium ★★★★★ Deque Index Simulation [Google](#)

Problem:

Order deck of cards so that revealing top card and moving next to bottom reveals in sorted order.

```
public int[] deckRevealedIncreasing(int[] deck) {
    int n = deck.length; Arrays.sort(deck);
    Deque<Integer> indexDeque = new ArrayDeque<>();
    for (int i = 0; i < n; i++) indexDeque.offer(i);
    int[] res = new int[n];
    for (int card : deck) {
        res[indexDeque.pollFirst()] = card; // Place smallest card
        if (!indexDeque.isEmpty()) {
            indexDeque.offerLast(indexDeque.pollFirst()); // Move next to back
        }
    }
    return res;
}
```

 **AHA MOMENT (LC 649):** Re-enqueueing the winning senator with an index offset 'idx + N' perfectly simulates round-robin voting!

HARD NEETCODE (SHORTEST SUBARRAY SUM & DP MAX) →

 HARD NEETCODE: SHORTEST SUBARRAY SUM & CONSTRAINED DP PAGE 9 OF 10

7. LEETCODE 862 — SHORTEST SUBARRAY WITH SUM AT LEAST K

Hard ★★★★★ Prefix Sum + Monotonic Deque [Amazon](#) [Google](#)

Problem:

Length of shortest non-empty subarray with sum >= K (handles negative numbers!).

```
public int shortestSubarray(int[] nums, int k) {
    int n = nums.length, minLen = n + 1;
    long[] P = new long[n + 1];
    for (int i = 0; i < n; i++) P[i + 1] = P[i] + nums[i];
    Deque<Integer> deque = new ArrayDeque<>();
    for (int i = 0; i ≤ n; i++) {
        while (!deque.isEmpty() && P[i] - P[deque.peekFirst()] ≥ k) {
            minLen = Math.min(minLen, i - deque.pollFirst());
        }
        while (!deque.isEmpty() && P[i] ≤ P[deque.peekLast()]) {
            deque.pollLast(); // Maintain monotonic increasing prefix sums
        }
        deque.offerLast(i);
    }
    return minLen ≤ n ? minLen : -1;
}
```

8. LEETCODE 1425 — CONSTRAINED SUBSEQUENCE SUM

Hard ★★★★★ DP + Monotonic Deque [Google](#)

Problem:

Max sum of subsequence such that index difference between consecutive elements is <= K.

```
public int constrainedSubsetSum(int[] nums, int k) {
    int n = nums.length, maxSum = nums[0];
    int[] dp = new int[n];
    Deque<Integer> deque = new ArrayDeque<>();
    for (int i = 0; i < n; i++) {
        if (!deque.isEmpty() && deque.peekFirst() < i - k) deque.pollFirst();
        dp[i] = nums[i] + (!deque.isEmpty() ? Math.max(0, dp[deque.peekFirst()]) : 0);
        maxSum = Math.max(maxSum, dp[i]);
        while (!deque.isEmpty() && dp[i] ≥ dp[deque.peekLast()]) deque.pollLast();
        deque.offerLast(i);
    }
    return maxSum;
}
```

 **FAANG COACH TIP:** Combining Dynamic Programming with a Monotonic Deque reduces $O(N \cdot K)$ DP transitions to $O(N)$ linear time!

TOPIC 08.0: QUEUE FOUNDATION & FIFO PRINCIPLE

Dry Run, Proofs & Cheat Sheet

MASTER REVISION & REFERENCE CHEAT SHEET →

◆ IF YOU REMEMBER ONLY ONE PAGE... MASTER REVISION SHEET

PAGE 10 OF 10

★ THE 4 GOLDEN LAWS OF QUEUES & DEQUES

1. **Use ArrayDeque for Queue & Deque:** Always declare `Deque<Integer> q = new ArrayDeque<>();` (Faster than LinkedList!).

2. **Monotonic Decreasing Deque for Sliding Max:** Pop smaller elements from BACK, pop expired indices from FRONT.

3. **Circular Queue Modulo Math:** Wrap indices via `(idx + 1) % capacity` to avoid O(N) element shifting.

4. **BFS Queue Snapshot:** For layer-by-layer tree/graph processing, freeze size via `int size = q.size();` before inner loop!

🚫 WHEN QUEUE FAILS (DO NOT USE!)

• **Random Index Access:** Queue ONLY allows viewing front element (O(1)) → Use **Array / ArrayList!**

• **LIFO Reversal:** Queue preserves order → Use **Stack** for LIFO reversal!

📖 NEETCODE & BLIND 75 CONFIDENCE TRACKER

Easy

LC 232 Queue using Stacks

Med

LC 622 Circular Queue

Hard

LC 239 Sliding Window Max

Easy

LC 933 Recent Calls

Med

LC 649 Dota2 Senate

Hard

LC 862 Shortest Subarray Sum

Easy

Moving Average Stream

Med

LC 950 Reveal Cards Order

Hard

LC 1425 Constrained Subset Sum

👉 Next Master Topic: Topic 09 → Priority Queue & Heap Masterclass

🏆 TOPIC 08 QUEUE & DEQUE MASTERCLASS COMPLETE! Turn the page to Topic 09: Priority Queue & Heap Masterclass!